

Evolución del sistema de automatización de cortinas inteligentes (Luflex) hacia protocolos MQTT

Facundo Boide

UNRaf, Universidad Nacional de Rafaela, Santa Fe, Argentina.
Ingeniería en computación 2 (IC)

Autor Corresp.: facundoboide@gmail.com

Abstract-- This technical report describes the technological evolution and architectural redesign of the Luflex automated window-covering prototype. The original system, developed as a standalone device with local Bluetooth control, was transformed into a distributed and scalable IoT solution. The new architecture integrates an ESP32 microcontroller reprogrammed for Wi-Fi wireless communication via the MQTT (Message Queuing Telemetry Transport) protocol, orchestrated over a microservice infrastructure based on Docker containers. The Eclipse Mosquitto messaging broker was incorporated alongside a Node-RED server for control flow management and a web user interface (Node-RED Dashboard). The document details the persistent electromechanical coupling, the new topic hierarchy, container metadata analysis, and dependency injection mechanisms for cross-platform deployments.

Resumen--El presente informe describe la evolución tecnológica y rediseño arquitectónico del prototipo de cerramiento automatizado Luflex. El sistema original, desarrollado como un dispositivo aislado con control via bluetooth, fue transformado en una solución IoT distribuida y escalable. La nueva arquitectura integra un microcontrolador ESP32 reprogramado para la comunicación inalámbrica Wi-Fi mediante el protocolo MQTT (Message queuing Telemetry Transport), orquestado sobre una infraestructura de microservicios basada en contenedores Docker. Se incorporo el sistema de mensajería Eclipse Mosquitto junto a un servidor Node-RED para la gestión de flujos de control e interfaz de usuario web (Node-RED Dashboard). El documento detalla el acoplamiento electromecánico persistente, la nueva jerarquía de tópicos, el análisis de metadatos de contenedores y los mecanismos de inyección de dependencias para despliegues multiplataforma.

Palabras clave: ESP32, MicroPython, MQTT, Cortina, Docker, Automatización, Mosquitto.

1. Introducción

En el ámbito de la automatización domiciliar, la transición desde dispositivos aislados con conectividad punto a punto hacia ecosistemas integrados en la Internet de las Cosas (IoT) resulta indispensable para garantizar escalabilidad y gestión remota (Azevedo et al., 2022). El proyecto Luflex fue concebido inicialmente como un prototipo de

automatización para cortinas controlado de forma local mediante el estándar Bluetooth Low Energy (BLE) e interaccionado a través de una aplicación móvil en MIT App Inventor (Boide & Gatti, 2026)[3]. Si bien la versión inicial satisfizo los requerimientos de control electromecánico y adquisición analógica de luz, la dependencia de una conexión de corto alcance limitaba la supervisión remota y la integración con otras plataformas domóticas. En esta segunda etapa de

desarrollo, correspondiente a la asignatura Introducción a los Computadores e Infraestructura IoT (ICII), el sistema fue reestructurado hacia un paradigma de microservicios. Se reemplazó el esquema de comunicación Bluetooth por el protocolo de mensajería liviano MQTT sobre la pila TCP/IP, virtualizando los servicios centrales mediante contenedores Docker para asegurar la portabilidad del entorno en hardware heterogéneo (Fielding & Taylor, 2020).

2. Estado del Arte

En el mercado actual de la domótica y los cerramientos inteligentes, las soluciones comerciales líderes —tales como Somfy, Tuya Smart, SwitchBot o Zemismart— han masificado la motorización de cortinas integrando conectividad hacia plataformas de terceros (Google Home, Amazon Alexa y Apple HomeKit). Sin embargo, la gran mayoría de estas alternativas comerciales operan bajo un modelo de arquitectura centralizada basado en la nube del fabricante (*Cloud-dependent*).

En los sistemas comerciales convencionales, la comunicación entre el dispositivo físico y la aplicación del usuario requiere una conexión constante hacia servidores externos propietarios a través de APIs de HTTP o REST. Este esquema presenta serias limitaciones respecto a la privacidad de datos,

dependencia de la infraestructura del proveedor, latencia operacional elevada ante problemas de conectividad a Internet y una vulnerabilidad crítica: si los servidores del fabricante quedan fuera de servicio, el control remoto del cerramiento deja de funcionar por completo.

Frente a esta problemática, las tendencias contemporáneas de la ingeniería IoT promueven el desarrollo de soluciones basadas en el protocolo **MQTT (Message Queuing Telemetry Transport)** desplegadas de forma local (*Local First*). Diseñado bajo una arquitectura ligera de Publicación/Suscripción sobre TCP/IP, MQTT permite que los dispositivos de borde (como el SoC ESP32) intercambien información de telemetría y comandos de forma asíncrona mediante un bróker local (Eclipse Mosquitto). A diferencia de las soluciones de mercado cerradas, la implementación de MQTT local ofrece las siguientes ventajas operacionales:

1. Soberanía y privacidad de datos: El flujo de información (lectura de LDRs y comandos de posicionamiento) no abandona la red local, eliminando riesgos de filtración hacia servidores de terceros.
2. Latencia reducida y bajo consumo: La trama binaria de MQTT requiere un encabezado mínimo (2 bytes), lo que optimiza el tráfico Wi-Fi local y ofrece tiempos de respuesta inmediatos comparados con las peticiones HTTP comerciales.

3. Interoperabilidad y estándar abierto: Mientras que los fabricantes imponen ecosistemas cerrados o licencias propietarias, MQTT utiliza tópicos estructurados compatibles de forma nativa con múltiples plataformas de orquestación como Node-RED o Home Assistant.
4. Independencia de Internet: El sistema Luflex mantiene su operatividad completa dentro de la red del hogar mediante el servidor central en Raspberry Pi, funcionando incluso si la red no cuenta con salida hacia la red pública de Internet.

3. Marco teórico

A- Arquitectura de Mensajería Orientada a Eventos: Protocolo MQTT:

El protocolo MQTT (Message Queuing Telemetry Transport) es un estándar abierto de red diseñado para el intercambio de mensajes livianos en entornos con restricciones de ancho de banda y latencia. A diferencia del patrón tradicional Petición/Respuesta propio de HTTP, MQTT opera bajo un esquema de Publicación/Suscripción (Pub/Sub) sobre la pila de transporte TCP/IP[4]. En esta topología, el microcontrolador de borde

(ESP32) y la plataforma de orquestación (Node-RED) interactúan como clientes independientes. La intermediación es gestionada de manera centralizada por un bróker (Eclipse Mosquitto), el cual se encarga de recibir, filtrar y enrutar las tramas de datos hacia los clientes suscriptos. La estructura de direccionamiento se organiza mediante una jerarquía de tópicos con delimitadores de barra (ej. facu/luflex/posicion), permitiendo un enrutamiento determinista de las variables de estado y comandos.

B- Virtualización a Nivel de Sistema Operativo: Contenerización mediante Docker:

La contenerización es una técnica de virtualización liviana que permite empaquetar una aplicación junto con sus bibliotecas, archivos de configuración y dependencias de entorno dentro de una unidad ejecutable aislada denominada contenedor. A diferencia de las máquinas virtuales tradicionales, los contenedores comparten el mismo núcleo (kernel) del sistema operativo anfitrión (host), lo que reduce drásticamente el consumo de recursos de cómputo y aceleró los tiempos de arranque. Para la orquestación de la plataforma se utiliza Docker Compose, una herramienta que permite definir y gestionar infraestructuras compuestas por múltiples microservicios mediante un único archivo de especificación declarativa (docker-compose.yml). Esta abstracción garantiza la portabilidad del entorno, aislando los datos volátiles de los persistentes a través del uso de volúmenes de almacenamiento[5].

C- Servidor Dedicado de Infraestructura (Raspberry Pi OS):

El alojamiento de la pila de servicios se ejecuta sobre una placa de cómputo, Raspberry Pi que corre el sistema operativo Raspberry Pi OS, cuenta con 4GB de memoria RAM, Bluetooth 5.0, BLE (bluetooth low energy), Ranura para tarjeta Micro-SD para cargar el sistema operativo y almacenar datos y más importante, cuatro núcleos Cortex-A72 (ARM v8) de 64 bits a 1,8 GHz[2]. Este nodo actúa como un servidor dedicado en modo *headless* (sin necesidad de periféricos gráficos o de entrada). La integración del motor de Docker dentro de los scripts de inicio del sistema operativo (*systemd/boot*) garantiza la resiliencia de la plataforma, asegurando la autorrecuperación y disponibilidad ininterrumpida de los servicios de mensajería y flujo ante cortes en el suministro eléctrico.

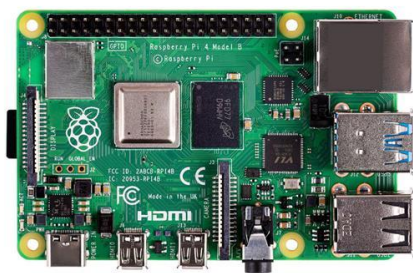


Ilustración 1 Raspberry Pi 4 Model B :

D- Entorno de Orquestación de Flujos e Interfaz de Usuario (Node-RED):

Node-RED es una herramienta de desarrollo basada en programación visual

desarrollada originalmente por IBM para la integración de dispositivos de hardware, APIs y servicios en el ámbito de la Internet de las Cosas. Construida sobre el entorno de ejecución Node.js, permite manipular eventos en tiempo real a través de nodos funcionales conectados en un lienzo de trabajo.

En la arquitectura del proyecto, Node-RED cumple una doble función: actúa como *engine* de reglas de negocio (procesando estados y determinando acciones automáticas) y como servidor de la interfaz web mediante el módulo *node-red-dashboard*, permitiendo la supervisión y control del cerramiento a través de cualquier navegador web dentro de la red local[6].

E- Capa de Instrumentación y Actuación Física (Resumen de Hardware):

La capa física del prototipo comprende la infraestructura electromecánica y sensorial desarrollada previamente[3]. Esta etapa incluye el SoC ESP32-WROOM como nodo de procesamiento local, un motor paso a paso NEMA 17 (17HS2408) con driver A4988 para la tracción mecánica, un sensor de efecto Hall A3144 para la determinación del punto cero de calibración (posición de cierre) y un arreglo de tres fotoresistencias LDR para la telemetría lumínica. La caracterización eléctrica y paramétrica detallada de esta instrumentación se encuentra documentada en la primera fase del

proyecto (Boide y Gatti, 2025).

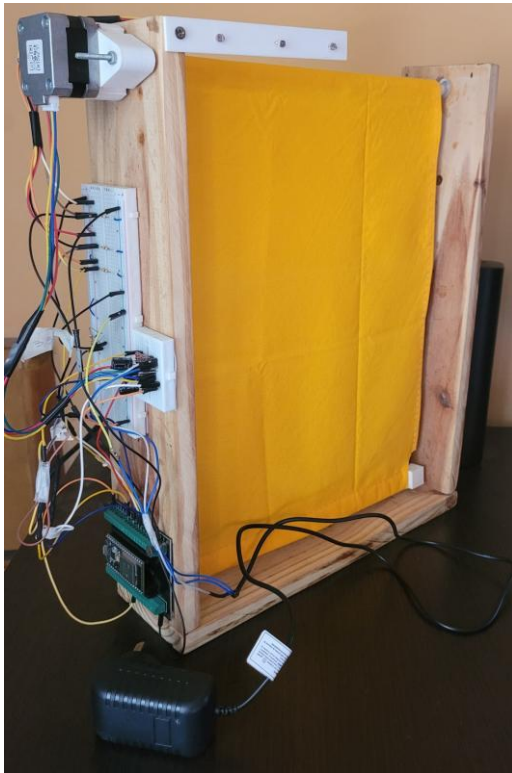


Ilustración 2: Maqueta Luflex

4. Desarrollo

El sistema Luflex constituye un dispositivo IoT de automatización edilicia para el control de la luz natural mediante un funcionamiento autónomo y asistido. La actuación electromecánica está a cargo de un motor paso a paso NEMA 17 (alimentado a 12 V / 2 A) accionado por un driver A4988. Para la adquisición de datos ambientales, un arreglo de tres fotoresistencias LDR en la parte superior exterior cuantifica la radiación solar. Todos los periféricos son gestionados por un microcontrolador SoC ESP32-WROOM [3].

Para garantizar la precisión posicional y evitar colisiones mecánicas al iniciar, el firmware en MicroPython ejecuta la función `calibrar_inicial()`. En esta secuencia, el motor se desplaza hacia un sensor de efecto Hall A3144 reduciendo su velocidad a la mitad (incrementando el retardo entre pasos de 2000 μ s a 4000 μ s) para detectar el punto cero magnético con exactitud milimétrica y registrar el conteo total de pasos del recorrido.

El sistema traduce los comandos y porcentajes de posición (0%, 50% o 100%) a desplazamientos físicos precisos basándose en la secuencia de calibración inicial realizada por la ESP32 al encenderse.

A partir de dicha referencia, el sistema toma decisiones sobre la posición objetivo de la persiana basándose en el valor analógico promediado de los tres sensores LDR conectados a la ESP32 (rango ADC de 0 a 4095), extendiendo la lógica desarrollada.

Las reglas de decisión según el modo seleccionado se estructuran de la siguiente manera:

1. Modo Automático 1 (Prioridad Luz Solar Directa):
 - $LDR < 1800$ (Baja luminosidad): Mantiene o mueve la cortina al 0% (Cerrada / 0 pasos).
 - $1800 \leq LDR < 3400$ (Luminosidad media): Ubica la cortina al 50% (Mitad del recorrido medido).
 - $LDR \geq 3400$ (Alta luminosidad): Desplaza la

- cortina al 100% (Apertura total / Recorrido máximo).
2. Modo Automático 2 (Prioridad Protección / Sombra):
 - $LDR > 3400$ (Exceso de sol): Mueve la cortina al 0% (Cerrada para proteger el ambiente/temperatura).
 - $1800 \leq LDR \leq 3400$ (Luminosidad media): Ubica la cortina al 50%.
 - $LDR < 1800$ (Baja luz): Abre la cortina al 100% para aprovechar el remanente de luz natural.

3. Modo Manual / Asistido:
 - Desactiva el algoritmo de decisión autónomo y permite al usuario definir la posición arbitraria de la cortina entre 0% y 100% mediante comandos directos ABRIR, CERRAR o valor porcentual específico vía slider, enviados desde el Dashboard.

La comunicación entre el ESP32 y el servidor Node-RED es asíncrona mediante el protocolo MQTT (puerto 1883) sobre Wi-Fi a través del bróker Eclipse Mosquitto. La capa de servidor se ejecuta en una placa Raspberry Pi en modo *headless*, cuyos microservicios están contenerizados mediante Docker Compose en la red virtual `luflex_net`. El archivo de

composición utiliza la directiva `restart: always` para asegurar la reanudación automática ante cortes de energía. Además, para automatizar el despliegue del Dashboard sin intervención manual, el contenedor de Node-RED se configuró con permisos `user: "root"` y una orden en el entrypoint que ejecuta `npm install node-red-dashboard` de forma previa al arranque de la aplicación.

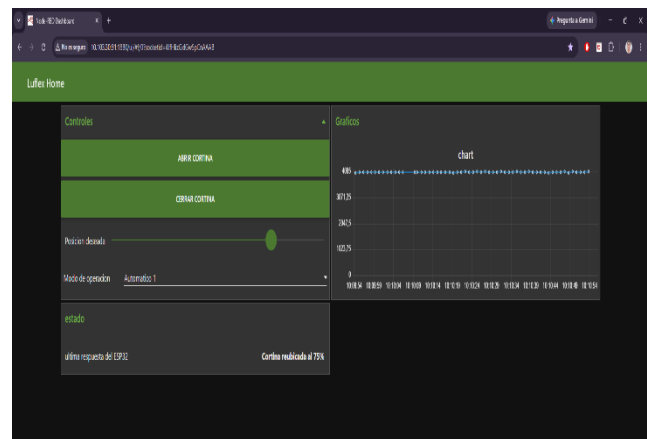


Ilustración 3: Aplicación Web con Node-Red

El intercambio de información se organiza bajo la raíz `facu/luflex/` mediante cinco tópicos:

- `facu/luflex/modo`: Node-RED publica los valores "manual", "auto1" o "auto2" desde un menú desplegable web. Al recibirlo, el ESP32 actualiza su estado e inactiva el control automático por luz si se selecciona el régimen manual.
- `facu/luflex/posicion`: Opera de forma bidireccional enviando valores enteros de "0" a "100" vinculados a un *Slider* interactivo. El ESP32 ejecuta `viajar_a_porcentaje()`, calculando los pasos proporcionales del motor. Asimismo, si la cortina se mueve de forma autónoma, el ESP32 publica en

este canal para actualizar la interfaz gráfica.

- `facu/luflex/comando`: Recibe las instrucciones "ABRIR" o "CERRAR" emitidas por botones web de acción rápida, derivando la orden directamente a los límites absolutos (100% o 0%).
- `facu/luflex/ldr`: El ESP32 publica cada 1000 ms un paquete JSON con el promedio de las tres LDRs. Node-RED procesa el dato y lo inyecta en un gráfico de línea temporal que dibuja la variación de luz segundo a segundo y en un indicador de nivel instantáneo.
- `facu/luflex/status`: Canal de retroalimentación donde el ESP32 publica mensajes de texto al finalizar un movimiento o calibración, actualizando un indicador de estado en la web que confirma la ejecución física del comando.

5. Conclusiones

La evolución tecnológica del prototipo Luflex desde un dispositivo aislado con control local vía Bluetooth hacia una solución distribuida basada en el protocolo MQTT y la virtualización con Docker consolida un modelo de arquitectura IoT altamente robusto, portable y escalable. La reestructuración del proyecto demostró que la migración hacia una topología *Local-First* orientada a eventos resuelve de manera eficiente las

limitaciones de alcance, latencia y dependencia de servidores externos propios de las soluciones comerciales convencionales, garantizando la soberanía de los datos dentro de la red local del usuario.

En la capa de instrumentación y procesamiento en el borde (*edge*), la reingeniería del firmware en MicroPython sobre el SoC ESP32 optimizó la gestión concurrente de tareas. La incorporación de la secuencia de calibración inicial a baja velocidad permitió compensar las holguras mecánicas del sistema y garantizar la precisión milimétrica en la determinación del punto cero mediante el sensor de efecto Hall A3144. Asimismo, el algoritmo de filtrado y promedio sobre el arreglo de sensores LDR aseguró un control lumínico estable, eliminando falsos disparos en el accionamiento del motor NEMA 17 frente a fluctuaciones solares transitorias.

Por su parte, el despliegue de la infraestructura de servidor central sobre la plataforma Raspberry Pi mediante Docker Compose representó un avance sustancial en términos de disponibilidad e integración. El aislamiento de los microservicios (Eclipse Mosquitto y Node-RED) en la red virtual `luflex_net`, junto con la política de reinicio automático (`restart: always`), dotó al sistema de una tolerancia a fallos crítica ante eventuales interrupciones del suministro eléctrico. La automatización del entorno se completó con éxito mediante la inyección directa de dependencias (`node-red-dashboard`) en el punto de entrada del contenedor, logrando un despliegue transparente de la interfaz gráfica y la sincronización

bidireccional de estados sin requerir intervención manual.

En síntesis, el proyecto Luflex-MQTT cumple holgadamente con los objetivos técnicos e integradores propuestos para la asignatura, validando un esquema conceptual replicable no solo para la automatización de cerramientos domiciliarios, sino también para aplicaciones de supervisión e infraestructura en el ámbito industrial.

6. Bibliografía

[1] Espressif Systems. (2023). ESP32 Series Datasheet

[Archivo PDF]. Disponible: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[2] Raspberry Pi. (2024). Specifications. [Sitio Web] Disponible: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/>

[3] Boide y Gatti (2025). Proyecto Luflex [Informe no publicado] Asignatura Ingeniería en Computación 1, Universidad Nacional de Rafaela.

[4] MQTT. (2019). MQTT: El estándar para la mensajería en el Internet de las Cosas (IoT) [Sitio WEB]. Disponible: <https://mqtt.org/>

[5] Dockerdocs. (2024). Docker compose [Sitio Web]. Disponible: <https://docs.docker.com/compose/>

[6] Node-Red(2026). Documentation [Sitio Web]. Disponible: <https://nodered.org/docs/>

7. Repositorio Git Hub

[7] Facundo Boide; página.

Disponible: <https://github.com/Facundo-Boide/202607-IC2-boide>